

what should the structure of a basic javascript quiz app be, which offers a question and returns feedback on the answer after each answer and after each 12 question round?

Here's a clean structure for a 12-question JavaScript quiz app:

Core Data

js

```
const questions = [  
  { question: "What does `typeof null` return?", options: ["null", "object", "undefined", "boolean"], answer: "object"  
    // ... 12 total  
};
```

State Object

js

```
const state = {  
  currentIndex: 0, // which question we're on  
  score: 0, // correct answers this round  
  selectedAnswer: null, // what the user picked  
  answered: false, // lock in after answering  
  round: 1, // track rounds if repeating  
};
```

Key Functions

Function	Responsibility
<code>loadQuestion()</code>	Render the current question + options to the DOM
<code>submitAnswer(selected)</code>	Check answer, update score, set <code>answered = true</code>
<code>showAnswerFeedback()</code>	Highlight correct/wrong, show explanation
<code>nextQuestion()</code>	Advance index, call <code>loadQuestion()</code>
<code>showRoundSummary()</code>	Triggered at question 12 — shows score + breakdown

Function

Responsibility

`resetRound()`

Reset state, optionally shuffle questions, restart

Flow

`loadQuestion()`

↓

User picks answer → `submitAnswer()`

↓

`showAnswerFeedback()` ← "✅ Correct! / ❌ Wrong — the answer was X"

↓

User clicks "Next" → `nextQuestion()`

↓

if index === 12 → `showRoundSummary()`

↓

User clicks "Play Again" → `resetRound()`

DOM Structure (HTML skeleton)

html

```
<div id="quiz">
  <div id="progress">Question 3 / 12</div>
  <div id="question-text"></div>
  <div id="options"></div>      <!-- buttons injected here -->
  <div id="feedback"></div>     <!-- per-answer feedback -->
  <button id="next-btn" hidden>Next → </button>
</div>

<div id="summary" hidden>
  <h2>Round Complete!</h2>
  <p id="final-score"></p>
  <div id="answer-breakdown"></div>
  <button id="restart-btn">Play Again</button>
</div>
```

A Few Design Tips

- **Lock options after answering** — disable all buttons once one is clicked to prevent changing the answer
- **Store answer history** — keep an array like `[{ question, selected, correct }]` so you can render a full breakdown at the end
- **Separate data from logic** — keep your questions array in its own file (`questions.js`) so it's easy to swap out
- **Shuffle optionally** — use a Fisher-Yates shuffle on the questions array at the start of each round for variety

Want me to build this out as a working app?

Don't tell me any code that should go into it, just how inputs and outputs should flow. I want you to teach me

Great approach — understanding the flow first makes the code much easier to write. Here's how inputs and outputs should move through the app:

The Two Phases

Every round has two repeating phases, plus a final summary phase:

Question Phase → **Answer Phase** → **(repeat 12x)** → **Summary Phase**

Question Phase

Input: Nothing from the user yet. The app reads the current question from your data based on the current index.

Output: The screen shows the question text, the answer options, and the progress indicator (e.g. "Question 3 of 12"). The next button is hidden.

Answer Phase

Input: The user clicks one of the answer options.

Output: Several things happen at once:

- All options become unclickable (so the answer is locked in)
 - The chosen option is visually marked as right or wrong
 - The correct answer is highlighted if they got it wrong
 - A feedback message appears explaining why
 - The next button becomes visible
-

Transition

Input: The user clicks Next.

Output: The app increments the question index. If the index is still under 12, it goes back to the Question Phase. If it just reached 12, it goes to the Summary Phase instead.

Summary Phase

Input: Nothing new from the user — the app reads from the history of answers it has been quietly building up throughout the round.

Output: The screen shows the final score, and a breakdown of every question — what the user answered and whether it was right or wrong.

Reset

Input: The user clicks Play Again.

Output: All state is wiped — index back to zero, score back to zero, history cleared — and the app returns to the Question Phase from the beginning.

The Key Insight

Notice that the user only ever does **two things**: click an answer, and click Next. Everything else is the app *reacting* to those two inputs by reading from its state and updating what's on screen. If you keep that separation clear in your code, the logic stays simple.

How should the javascript be placed in the html code for this app? should it be in a separate document like a css file? or amongst the html in some way?

Great question, and the answer depends on the scale of your project. Here's how to think about it:

The General Rule

JavaScript should almost always live in a **separate .js file**, linked at the bottom of your HTML. This mirrors exactly how you'd treat CSS — keep concerns separated.

```
quiz/  
  index.html  
  style.css
```

```
app.js
questions.js
```

Where in the HTML to Link It

Put your `<script>` tags at the **bottom of the `<body>`**, not in the `<head>`. Like this:

html

```
<body>
  <!-- all your html content -->

  <script src="questions.js"></script>
  <script src="app.js"></script>
</body>
```

The reason is simple — your JavaScript will need to find and manipulate HTML elements. If the script runs before those elements exist in the page, it won't find them. Placing scripts at the bottom guarantees the HTML is fully loaded first.

Why Two Separate JS Files Here

For a quiz app, splitting into `questions.js` and `app.js` is worth it because they do completely different jobs:

- **questions.js** is just data — your array of questions, answers, and explanations
- **app.js** is logic — all the functions that make the app actually work

This means you could swap out the entire question set without touching any of your logic, and vice versa.

When Inline JS Makes Sense

Inline `<script>` tags inside the HTML body are generally considered bad practice for an app like this. The one exception is a tiny, genuinely throwaway script — but for anything you're building to learn from or grow, keep it external.

The Bigger Picture

The separation you're working towards is:

- **HTML** — structure (what's on the page)
- **CSS** — appearance (what it looks like)
- **JS** — behaviour (what it does)

Each file should be ignorant of the others as much as possible. Your quiz app is a perfect project to practice that discipline from the start.

What should I use to present the question in? It will be a different question each time, but it should stay in the same place, governed by CSS, so it will be called something like `QuestionBox`. It should be able to be cleared and refilled every time an answer is submitted. I'll tell you later how it will be filled, for now just tell me what the question should appear in.

Use a `<div>` with an `id` of something like `question-box`.

A `<div>` is the right choice here because it's a generic container with no built-in meaning or behaviour — it just holds whatever you put inside it, sits where CSS tells it to sit, and can be completely cleared and rewritten by JavaScript at any time.

The `id` is what lets your JavaScript find it instantly and your CSS style it precisely.

And I can just outline it using CSS, right?

Yes, exactly. You can give it a border, rounded corners, background colour, padding, width, position on the page — whatever you like. The `id` you gave it is all CSS needs to target it and make it look however you want.

How should the JavaScript be structured so that single cells from a specific column in a table are inserted into `<p id="Question">` in `<div id="QuestionBox">` when the quiz is moved on from the previous, answered question?

Remember the flow we talked about — your JavaScript has a **current index** that tracks which question you're on. When the user moves to the next question, the index increments, and then a function reads from your questions data using that index and puts the right content into the page. For your specific setup it would work like this:

The Data

Your questions table (array of objects) has a column (property) for the question text, something like `questionText`. Each row is one question.

The Function

You'd have a `loadQuestion()` function that:

1. Reads the current index
2. Goes to that row in your data
3. Pulls out just the `questionText` cell from that row
4. Finds `<p id="Question">` in the DOM
5. Sets its content to that text, replacing whatever was there before

The Trigger

That function gets called in two situations:

- Once when the app first loads, to show question one
- Again every time the user clicks Next, after the index has already been incremented

The Key Point

The `<p>` tag doesn't move, doesn't get destroyed, and doesn't get recreated. JavaScript just reaches into it and overwrites its text content each time. CSS never has to care — it's always styling the same element in the same place. Only the words inside it change.

That single pattern — **find the element, overwrite its content** — is essentially all your `loadQuestion()` function needs to do for the question text.

Teach me, without showing me an example of code, what concept I should use to retain the answer of each question as a wrong/right binary? You can tell me what it's called, just don't show me an example

The concept you want is an **array** that you **push** to after each answer is submitted.

Every time the user answers a question, you push a single true or false value into the array — true if they got it right, false if they didn't. By the end of the round, you have an array of 12 true/false values, one for each question in order.

This is useful in two ways:

- **The score** is just a count of how many `true` values are in the array
- **The breakdown** at the end of the round is the array itself — you go through it one by one and know exactly which questions were right and which were wrong

The array starts empty at the beginning of each round, and gets reset to empty again when the round resets. It grows by one entry each time an answer is submitted, so by question 12 it's full.

This concept is sometimes called an **answer history** or **results log** — just an ordered record of what happened, built up gradually as the app runs.

Before the array is used at the end, I will want to mark the answer given—highlighting the correct answer green and a wrong answer red, with CSS. Will the array be used for that?

No — the array isn't needed for that part at all.

That per-question feedback happens immediately, in the moment, before anything gets pushed to the array. Your JavaScript already knows two things the instant the user clicks an answer:

- Which option the user clicked
- What the correct answer is for the current question

That's all it needs to apply the right CSS highlighting then and there. It doesn't need to look anything up or remember anything from before.

The array is only for **looking back** — it's for the summary at the end of the round when you need a record of all 12 answers together. It serves a completely different purpose to the immediate feedback.

In this code:

```
const person = ["John", "Doe", 46];
```

```
const person = {firstName:"John", lastName:"Doe", age:46};
```

How does the javascript know that Doe is john doe's last name and not a separate entity?

In the first line, it doesn't. "Doe" is just the second item in a list — JavaScript has no idea what it means. It only knows it's at position 1. The meaning exists only in your head as the programmer.

In the second line, it knows because "Doe" has been given a **label** — `lastName`. The value and its meaning are packaged together as a pair. JavaScript doesn't interpret the word "lastName" in any human sense, but it knows that "Doe" belongs to that named property on that object, and nothing else.

That's the fundamental difference between the two — an array is an ordered list of values, an object is a collection of named values. For your quiz data, this is exactly why objects are the right choice. A question, its options, and its correct answer aren't just a list of separate things — they're related pieces of information that belong together under named labels.

